

PR comment draft

I narrowed the reentrant publish bypass so it only applies to snapshot injection from `requestSnapshot()`, rather than to any nested publish while `#isPublishing` is true.

Concretely:

- `#onMessages(...)` and `#publish(...)` now accept an internal opt-in flag for reentrant bypass
- ordinary stream batches, including SSE batches, still serialize through `#messageChain`
- only `requestSnapshot()` calls `#onMessages(..., { allowReentrantPublishBypass: true })`

This preserves the original deadlock fix:

- if a subscriber handling batch M1 does `await requestSnapshot(...)`, the injected snapshot batch can still publish immediately instead of being queued behind the subscriber that is awaiting it

But it avoids the broader regression:

- later SSE batches no longer bypass earlier in-flight publishes just because `#isPublishing === true`

I added tests for both sides of the behavior:

1. SSE ordering regression test

- proves a later SSE batch is not delivered before subscriber callbacks for the earlier batch complete

2. Bystander subscriber behavior test

- explicitly documents current behavior that a reentrant `requestSnapshot()` may re-enter bystander subscribers before their earlier callback completes

So the resulting contract is:

- **ordinary stream traffic remains serialized**
- **snapshot injection is the only allowed reentrant bypass**
- **bystander reentrancy during snapshot injection remains allowed and now has test coverage**